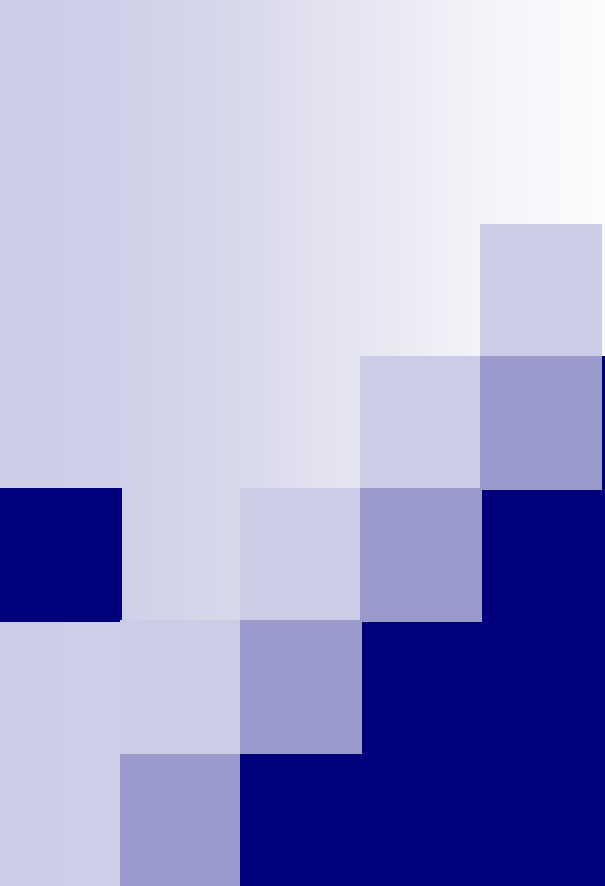




Laboratorio di BD1

Il linguaggio SQL

Lezione 1

Docente: Daniele Riboni
Tutor: Fabio Pili e Luca Garau

Dipartimento di Matematica e Informatica
Università degli Studi di Cagliari

Il linguaggio SQL

- È il linguaggio standard per la definizione, manipolazione ed interrogazione delle basi di dati relazionali.
- È utilizzato da tutti i DBMS relazionali attualmente disponibili sul mercato.
- Il nome SQL è l'acronimo di “Structured Query Language”



Cosa impareremo oggi

- Impareremo a creare uno schema...
 - ... a creare dei tipi di dato ...
 - ... a creare delle tabelle e definire vincoli ...
 - ... usando il linguaggio SQL
- Impareremo ad usare PostgreSQL e a fargli eseguire i nostri comandi SQL

SQL: un po' di storia

- Prima proposta: SEQUEL (IBM Research, 1974)
- Prima implementazione commerciale in SQL/DS (IBM, 1981)
- “Standard de facto” dal 1983
- Lungo processo di standardizzazione nell'ambito di ANSI (*American National Standards Institute*) e ISO (*International Standards Organization*)

SQL: standardizzazione

- Prima versione ufficiale nel 1986 (**SQL-86**), rivista ed estesa nel 1989 (**SQL-89**)
- Seconda versione nel 1992 (**SQL-2** o **SQL-92**)
- Terza versione: nel 1999 (**SQL-3** o **SQL:1999**), rivista ed estesa nel 2003 (**SQL:2003**), nel 2006 (**SQL:2006**), nel 2008 (**SQL:2008**), e nel 2011 (**SQL:2011**)

SQL: standardizzazione

- SQL-86

- costrutti base

- SQL-89

- integrità referenziale

- SQL-92

- nuovi costrutti (es: operatore di join)
 - tre livelli di adesione: *entry*, *intermediate*, *full*

SQL: standardizzazione

■ SQL:1999

- pienamente compatibile con SQL-92
- funzionalità avanzate:
supporto per il paradigma a oggetti,
viste ricorsive, trigger, etc.
- organizzazione modulare:
nucleo + pacchetti specializzati opzionali

SQL: standardizzazione

■ SQL:2003

- eliminazione di costrutti non utilizzati
- estensioni del modello a oggetti
- integrazione con XML, metodi Java, etc.

■ SQL:2006, SQL:2008, SQL:2011

- ulteriori funzionalità per dati XML, supporto a dati temporali complessi

SQL: adesione allo standard

- I diversi sistemi relazionali disponibili sul mercato propongono implementazioni differenti e incomplete del linguaggio SQL:
 - le differenze si osservano in particolare nelle funzionalità più innovative
 - vedi <http://troels.arvin.dk/db/rdbms/> per un confronto

SQL: principali funzionalità

- DDL (Data Definition Language)
 - *definizione di schemi relazionali*
- DML (Data Manipulation Language)
 - *aggiornamento e interrogazione delle istanze*
- DCL (Data Control Language)
 - *controllo dell'accesso ai dati*

Definizione dei dati in SQL

- Il comando SQL fondamentale per la definizione dei dati è l'istruzione **CREATE**, che può essere usata per creare schemi, tabelle, domini e altri oggetti.
- Nota: SQL usa i termini
tabella, **riga** e **colonna**
in luogo, rispettivamente, dei termini
relazione, *tupla* e *attributo*
del modello relazionale.

Esempio: definizione di una tabella

- Esempio:

*char(n) / varchar(n) sono
tipi stringa a lunghezza
fissa / variabile*

```
CREATE TABLE Impiegato
( Matricola CHAR(6) PRIMARY KEY,
  Nome VARCHAR(20) NOT NULL,
  Cognome VARCHAR(20) NOT NULL,
  Dipartimento VARCHAR(15),
  Stipendio DECIMAL(10,2) DEFAULT 0,
  FOREIGN KEY(Dipartimento)
  REFERENCES Dipartimento(NomeDip),
  UNIQUE (Cognome, Nome)
)
```

Tipi di dato

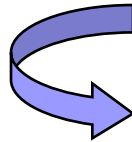
*oggi ne vedremo
solo alcuni...*

- tipi predefiniti:
 - tipi numerici (esatti e approssimati)
 - tipi carattere (singoli caratteri o stringhe)
 - tipi temporali (data, ora, intervalli di tempo)
 - tipi booleani

- tipi user-defined

Assegnamento a *NULL*

- È compatibile con tutti i tipi di dato.
- Non è un valore. Rappresenta assenza di informazione.



Si usa per attributi non applicabili o dal valore non noto.

Assegnamento a NULL

Iscrizione Palestra

Nome* :

Cognome * :

Email :

Telefono :

Via* :

Città* :

I campi contrassegnati con l'asterisco sono obbligatori.

Nome	Cognome	Email	Telefono	Via	Citta
Franco	Cau	fc@live.it		Francia, 9	Cagliari
Paolo	Ferru	pf@gmail.com	3468256340	Carducci, 10	Cagliari
Sara	Rossi			Padova, 71	Roma

Tipi booleani

■ BOOLEAN

- rappresenta i valori booleani

TRUE, FALSE, UNKNOWN

(logica a tre valori)

- il valore UNKNOWN è stato introdotto in SQL per la gestione dei confronti con NULL

Tipi booleani

Cittadini

Nome	Cognome		Via	Città	Professione	Stipendio _annuo
Franco	Cau	...	Francia, 9	Cagliari	Studente	NULL
Paolo	Ferru	...	Carducci, 10	Cagliari	Disoccupato	NULL
Sara	Rossi	...	Padova, 71	Roma	Impiegato	30000

```
SELECT nome, cognome  
FROM cittadini  
WHERE stipendio_annuo > 25000;
```

Franco Cau **unknown**
Paolo Ferru **unknown**

NULL e UNKNOWN

- Quando un valore NULL è coinvolto in un confronto, il risultato è UNKNOWN.
- Quando un valore NULL è coinvolto in un'operazione algebrica, il risultato è NULL.
- Ciascun valore NULL è considerato, in generale, differente da ogni altro NULL.

Tipi booleani

Il valore UNKNOWN

- Esempio di interrogazione: “Selezionare tutti gli impiegati che *lavorano al dipartimento di Informatica e hanno stipendio maggiore a 20.000 euro*”
- Supponiamo che Mario Rossi lavori al dipartimento di Informatica e il suo stipendio sia NULL
- Come lo valutiamo?

Logica a tre valori in SQL

AND	true	false	unknown
true	true	false	unknown
false	false	false	false
unknown	unknown	false	unknown

OR	true	false	unknown
true	true	true	true
false	true	false	unknown
unknown	true	unknown	unknown

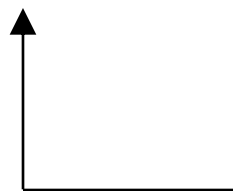
NOT	
true	false
false	true
unknown	unknown

Tipi predefiniti: Definizione di domini

- A partire dai domini predefiniti,
 è possibile definire nuovi domini
 da associare agli attributi di uno schema:

CREATE DOMAIN NomeDominio

AS TipoDiDato [ValoreDiDefault] [Vincoli]



dominio elementare (predefinito o
definito dall'utente in precedenza)

Notazione

- [...] □ per indicare un termine opzionale
(che può comparire al più una volta)
- {...} □ per indicare un termine che può non comparire o essere ripetuto più volte
- **termine₁ | termine₂ | | termine_n**
 - per indicare un elenco di termini in alternativa

(N.B. elenchi consecutivi possono essere delimitati dalle parentesi angolari <...>)²²

Definizione di domini

*2 bytes,
range: -32768 to +32767*

■ Esempi:



```
CREATE DOMAIN Voto AS SMALLINT  
DEFAULT NULL  
CHECK ( value >=18 AND value <= 30 )
```

```
CREATE DOMAIN AnniCorso AS SMALLINT  
DEFAULT 3  
NOT NULL
```

Esercizio

- Modificando l'esempio precedente, creare un dominio CodiceFiscale di tipo testuale, imponendo che la lunghezza della stringa ad esso assegnata sia 16
- N.B.: se assegnamo ad una colonna di tipo CHAR(n) una stringa di lunghezza inferiore a n , vengono aggiunti automaticamente degli spazi in fondo alla stringa per arrivare a n caratteri
- Suggerimento: Usare nel vincolo CHECK la funzione LENGTH(*value*) che restituisce la lunghezza della stringa *value*

```
Esempio precedente:  
CREATE DOMAIN Voto  
AS SMALLINT DEFAULT NULL  
CHECK ( value >=18 AND value <= 30 )
```

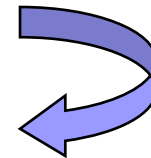

Soluzione

- Modificando l'esempio precedente, creare un dominio CodiceFiscale di tipo testuale, imponendo che la lunghezza della stringa ad esso assegnata sia 16

```
CREATE DOMAIN CodiceFiscale  
AS CHAR(16)  
CHECK (LENGTH(value)=16)
```

Perché creare domini personalizzati?

- Riuso delle definizioni
- Schemi più comprensibili
e facilmente modificabili



*Se si modifica la definizione di un dominio,
la modifica viene propagata in tutte
le tabelle in cui il dominio viene usato.*

Definizione di uno schema

- SQL consente la definizione di uno schema di basi di dati come collezione di oggetti (domini, tabelle, viste, etc):

***CREATE SCHEMA [NomeSchema]
[AUTHORIZATION NomeProprietario]
{ DefElementoSchema }***

Definizione di uno schema

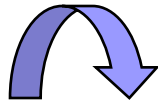
- Esempio:

```
CREATE SCHEMA AZIENDA  
AUTHORIZATION JSMITH
```

- Se *AUTHORIZATION NomeProprietario* viene omissso, si assume come proprietario l'utente che ha lanciato il comando.
- Se *NomeSchema* viene omissso, si assume come nome dello schema il nome del proprietario.

Definizione di uno schema

- Non tutti gli utenti sono autorizzati a creare gli schemi e i rispettivi elementi.



Il privilegio di creare schemi, tabelle e altri oggetti deve essere esplicitamente attribuito dall'amministratore del sistema agli account utente per cui è necessario.

Definizione delle tabelle

- L'istruzione **CREATE TABLE**:
 - definisce lo schema di una tabella
e ne crea un'istanza vuota
 - specifica attributi, domini,
valori di default e vincoli
- Il creatore possiede tutti i privilegi sulla tabella,
cioè i diritti di accedere ai dati e di modificarli.

Definizione delle tabelle

- Sintassi:

```
CREATE TABLE NomeTabella  
( NomeColonna Dominio [ ValoreDiDefault ]  
                                [ VincoliColonna ],  
.....,  
NomeColonna Dominio [ ValoreDiDefault ]  
                                [ VincoliColonna ],  
[ AltriVincoli ]  
)
```

Definizione delle tabelle

- Esempio:

```
CREATE TABLE Impiegato
( Matricola CHAR(6) PRIMARY KEY,
  Nome VARCHAR(20) NOT NULL,
  Cognome VARCHAR(20) NOT NULL,
  Dipartimento VARCHAR(15),
  Stipendio DECIMAL(10,2) DEFAULT 0,
  FOREIGN KEY(Dipartimento)
  REFERENCES Dipartimento(NomeDip),
  UNIQUE (Cognome, Nome)
)
```

*char(n) / varchar(n) sono
tipi stringa a lunghezza
fissa / variabile*

Definizione delle tabelle

- Solitamente lo schema SQL in cui vengono dichiarate le tabelle è specificato implicitamente dall'ambiente in cui sono eseguite le istruzioni CREATE TABLE.
- Si può esplicitamente associare il nome dello schema ai nomi delle relazioni:

CREATE TABLE NomeSchema.NomeTabella

Esempio:

CREATE TABLE AZIENDA.Impiegato

Definizione delle tabelle

- Le relazioni dichiarate attraverso l'istruzione CREATE TABLE sono chiamate **tabelle di base** (o **relazioni di base**)
 - la relazione e le sue tuple sono effettivamente create e memorizzate come un file dal DBMS
- Le relazioni di base si distinguono dalle **relazioni derivate**, create attraverso l'istruzione CREATE VIEW, che possono corrispondere o no a un file fisico.

Definizione delle tabelle

■ **N.B.**

In SQL

- le colonne di una tabella sono considerate ordinate secondo la sequenza specificata nell'istruzione CREATE TABLE
- le righe (tuple) non sono considerate ordinate

Valori di default

- Sintassi per la specifica dei valori di default:

DEFAULT < GenericoValore | USER | NULL >

dove:

- *GenericoValore* rappresenta un valore compatibile con il dominio (una costante o il risultato di un'espressione)
- *L'opzione USER* impone come default l'identificativo dell'utente che esegue il comando di aggiornamento
- *L'opzione NULL* corrisponde al valore di default di base

Valori di default

- Esempi:

NumeroFigli SMALLINT DEFAULT 0

Nome VARCHAR(20) DEFAULT USER

Esercizio

- Creare tre tabelle per la gestione dei pesci presenti nelle diverse vasche di un acquario:
 - Una tabella Pesce con campi per il nome, la famiglia, ed un codice numerico del tipo di pesce
 - Una tabella Vasca per il nome e il codice della vasca, il suo volume in litri, e il nome della sala in cui si trova
 - Una tabella Presenza per tenere traccia della quantità dei vari tipi di pesci presenti nelle diverse vasche

- Esempio:
 - `CREATE TABLE Pesce (nome VARCHAR(20), famiglia VARCHAR(20), codicep SMALLINT);`

Soluzione

```
CREATE TABLE Vasca (  
    codicev SMALLINT,  
    volume SMALLINT,  
    sala VARCHAR(15),  
    nomevasca VARCHAR(20));
```

```
CREATE TABLE presenza (  
    codicevasca SMALLINT,  
    codicepesce SMALLINT,  
    quantita SMALLINT;
```

Vincoli di integrità

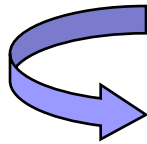
- Vincoli che possono essere specificati in SQL come parte della creazione di una tabella:
 - **vincoli intra-relazionali:**
 - vincoli di (super)chiave
 - vincoli di dominio
 - vincoli su singole tuple
 - **vincoli inter-relazionali:**
 - vincoli di integrità referenziale

Vincoli intra-relazionali

- Specificano condizioni sui valori di una *singola tabella*:
 - **NOT NULL**
 - **UNIQUE**
 - **PRIMARY KEY**
 - **CHECK**

Vincoli intra-relazionali

■ **NOT NULL**



Indica che un attributo
non può assumere
valori nulli

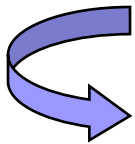
Esempi:

Cognome VARCHAR(20) **NOT NULL**

Data_nascita DATE **NOT NULL**

Vincoli intra-relazionali

■ **UNIQUE**



Definisce una (super)chiave:

- due righe della tabella non possono avere gli stessi valori sull'attributo (o l'insieme di attributi) cui si applica il vincolo
- viene fatta eccezione per il valore NULL

Vincoli intra-relazionali

- Il vincolo UNIQUE può essere definito in due modi:

1) Si fa seguire la specifica dell'attributo dalla parola chiave UNIQUE

(N.B. solo quando il vincolo
si applica ad un singolo attributo)

Esempio:

Matricola CHAR(6) **UNIQUE**

Vincoli intra-relazionali

Esempio:

Matricola CHAR(6) **UNIQUE**

Matricola	Nome	Cognome	
90564	Franco	Collu	..
90564	Franco	Pinna	..
98763	Sara	Orru	...
76523	Sara	Orru	...

Vincoli intra-relazionali

- 2) Si usa il costrutto **UNIQUE** (ListaAttributi) dopo aver definito *tutti* gli attributi della tabella:

```
CREATE TABLE Impiegato
( Matricola CHAR(6) PRIMARY KEY,
  Nome VARCHAR(20) NOT NULL,
  Cognome VARCHAR(20) NOT NULL,
  .....
  → UNIQUE (Cognome, Nome)
)
```

Vincoli intra-relazionali

```
CREATE TABLE Impiegato
( Matricola CHAR(6) PRIMARY KEY,
  Nome VARCHAR(20) NOT NULL,
  Cognome VARCHAR(20) NOT NULL,
  .....,
  UNIQUE (Cognome, Nome)
)
```

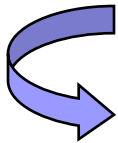
Matricola	Nome	Cognome	
90564	Franco	Collu	..
87541	Pietro	Pinna	..
98763	Sara	Orru	...
76523	Sara	Orru	...

Vincoli intra-relazionali

N.B. Il vincolo **UNIQUE** (Cognome, Nome)
impone che non ci siano due righe che abbiano uguali
sia il nome che il cognome ...

... ed è ben diverso da

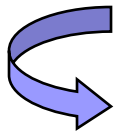
Nome VARCHAR(20) NOT NULL **UNIQUE**,
Cognome VARCHAR(20) NOT NULL **UNIQUE**



In questo caso, si ha una violazione
se nella tabella compaiono più di una volta
o lo stesso nome o lo stesso cognome.

Vincoli intra-relazionali

■ PRIMARY KEY



Definisce la **chiave primaria**

Implica NOT NULL

- ❑ gli attributi su cui è definito il vincolo PRIMARY KEY non possono assumere il valore nullo

Vincoli intra-relazionali

- Il vincolo PRIMARY KEY, come il vincolo UNIQUE, può essere definito:

- *direttamente su un singolo attributo:*

Matricola CHAR(6) PRIMARY KEY

- *specificando la lista di attributi a cui si applica il vincolo:*

Nome VARCHAR(20) ,
Cognome VARCHAR(20),
.....,
PRIMARY KEY (Cognome, Nome)

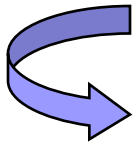
Vincoli intra-relazionali

- **N.B.** In una tabella è possibile specificare più chiavi UNIQUE ma una sola PRIMARY KEY

```
CREATE TABLE Impiegato
( Matricola CHAR(6) PRIMARY KEY,
  Nome VARCHAR(20) NOT NULL,
  Cognome VARCHAR(20) NOT NULL,
  CF CHAR(16) NOT NULL UNIQUE,
  UNIQUE (Cognome, Nome)
)
```

Vincoli intra-relazionali

- **CHECK (*Condizione*)**



Esprime un generico vincolo
tramite un'espressione
che deve essere vera
per tutte le tuple della tabella

Vincoli intra-relazionali

- I vincoli CHECK che coinvolgono un singolo attributo possono essere specificati subito dopo la definizione dell'attributo:

Mansione VARCHAR(20) **CHECK** (Mansione IN ('dirigente', 'ingegnere', 'tecnico', 'segretaria'))

Voto INTEGER **CHECK** (Voto >= 18
AND Voto <= 30)

Vincoli intra-relazionali

- I vincoli CHECK che coinvolgono più attributi devono essere specificati dopo aver definito *tutti* gli attributi:

```
CREATE TABLE Impiegato
(  Matricola CHAR(6) PRIMARY KEY,
   .....
   Stipendio DECIMAL(10,2) DEFAULT 0,
   Premio DECIMAL(10,2) DEFAULT 0,
   → CHECK (Stipendio > Premio)
)
```

Constraints

- È possibile assegnare un nome ai vincoli usando la parola chiave **CONSTRAINT**.

Esempio:

CONSTRAINT *StipOk*
CHECK (Stipendio > Premio)

Nome
vincolo


- È possibile aggiungere/rimuovere constraint alle tabelle